Iniciado em	terça-feira, 7 jan. 2025, 14:00
Estado	Finalizada
Concluída em	terça-feira, 7 jan. 2025, 15:33
Tempo empregado	1 hora 33 minutos
Avaliar	**9,80** de um máximo de 10,00(**98**%)

Questão 1

Correto

Atingiu 1,00 de 1,00

Dado a seguinte definição para uma fila:

```
typedef struct {
    int dado;
    struct No* prox;
} No;

typedef struct {
    No* inicio;
    No* fim;
    int tamanho;
} Fila;
```

Qual é a implementação correta para a função ***dequeue***?

a.
```
int dequeue(Fila* l) {
    if (l->inicio == NULL) return 0;

    No* primeiro = l->inicio;
    l->inicio = l->inicio->prox;
    free(primeiro);
    l->tamanho--;
    return 1;
}
```
✔ Parabéns!

b.
```
int dequeue(Fila* l) {
    if (l->inicio == NULL) return 0;

    No* primeiro = l->inicio;
    l->inicio = l->inicio->prox;
    l->tamanho--;
    return 1;
}
```

c.
```
int dequeue(Fila* l) {
    if (l->inicio == NULL) return 0;

    No* ultimo = l->fim;
    l->fim = l->fim;
    free(ultimo);
    l->tamanho--;
    return 1;
}
```

d.
```
void dequeue(Fila* l, int item) {
    No* novo = (No*) malloc(sizeof(No));
    novo->dado = item;
    novo->prox = NULL;
    if (l->fim != NULL) {
        l->fim->prox = novo;
    } else {
        l->inicio = novo;
    }
    l->fim = novo;
    l->tamanho++;
}
```

e.

```
void dequeue(Fila* l, int item) {
    No* novo = (No*) malloc(sizeof(No));
    novo->dado = item;
    novo->prox = NULL;
    if (l->inicio != NULL) {
        l->inicio->prox = novo;
    } else {
        l->inicio = novo;
    }
    l->inicio = novo;
    l->tamanho++;
}
```

Sua resposta está correta.

Questão 2

Correto

Atingiu 2,00 de 2,00

As estruturas de dados são fundamentais na programação, pois permitem armazenar, organizar e manipular informações de maneira eficiente. Três dessas estruturas são listas, filas e **pilhas**.

Lista

Uma lista é uma estrutura de dados [genérica] ✔ que armazena elementos de forma [encadeada] ✔, onde cada elemento pode ser acessado diretamente por sua [posição] ✔. Em uma lista encadeada, cada elemento é armazenado em um [nó] ✔, que contém o valor e um [ponteiro] ✔ para o próximo elemento.

Fila

Uma fila é uma estrutura de dados que segue o princípio [FIFO (First In, First Out)] ✔, ou seja, o primeiro elemento que entra é o [primeiro] ✔ a sair. As operações básicas de uma fila são **enqueue**, para [inserir] ✔ um elemento, e **dequeue**, para [remover] ✔ um elemento. Um exemplo prático de fila é a fila de [pessoas no supermercado] ✔.

Pilha

Uma pilha é uma estrutura de dados que segue o princípio [LIFO (Last In, First Out)] ✔, ou seja, o [último] ✔ elemento que entra é o primeiro a sair. As operações básicas de uma pilha são **push**, para [inserir] ✔ um elemento no topo, e **pop**, para [remover] ✔ o elemento do topo. Um exemplo prático de pilha é uma pilha de [pratos empilhados] ✔.

Essas estruturas desempenham papéis importantes em algoritmos e sistemas computacionais, sendo utilizadas em diversas áreas, como gerenciamento de memória, resolução de problemas e organização de dados.

Sua resposta está correta.

Questão 3

Parcialmente correto

Atingiu 1,80 de 2,00

A alocação dinâmica de memória permite ao programador maior flexibilidade e economia de memória utilizada pelo seu programa. Considere as declarações do registro *Profissao* e a assinatura da função **aloca_profissao** abaixo.

```
typedef struct{
    int codigo;
    char *nome;
} Profissao;

Profissao* aloca_profissao(unsigned int tamanho_nome);
```

A função **aloca_profissao** recebe como parâmetro o tamanho do nome da profissão, e deve alocar dinamicamente um registro para ***Profissao*** e também o espaço adequado para armazenar o nome de forma dinâmica, retornando o endereço para o registro criado. Nos testes, o tamanho do nome é sempre positivo (e maior que zero), e será lido automaticamente da entrada padrão. Implemente esta função, respeitando a assinatura especificada, conforme descrição do seu comportamento apresentada.

Observação: nesta questão é obrigatório o uso da função `malloc`.

For example:

Test	Input	Result
`printf("%lu\n", bytes_alocados());`	10	26
`printf("%lu\n", bytes_alocados());`	40	56

Answer: (penalty regime: 0, 0, 10, 20, ... %)

RESET ANSWER

```
Profissao* aloca_profissao(unsigned int tamanho_nome){
    Profissao *p = (Profissao*) malloc(sizeof(Profissao));
    p->nome = (char*) malloc(tamanho_nome * sizeof(char));
    if (p->nome == NULL) {
        free(p);
        exit(1);
    }
    return p;
}
```

	Test	Input	Expected	Got	
✔	`printf("%lu\n", bytes_alocados());`	10	26	26	✔
✔	`printf("%lu\n", bytes_alocados());`	40	56	56	✔
✔	`printf("%lu\n", bytes_alocados());`	100	116	116	✔
✔	`printf("%lu\n", bytes_alocados());`	42	58	58	✔
✔	`printf("%lu\n", bytes_alocados());`	123	139	139	✔

Your code failed one or more hidden tests.

malloc é uma função bem interessante, ainda mais que seu funcionamento nem sempre é garantido.

Parcialmente correto

Notas para este envio: 1,80/2,00.

Questão 4

Correto

Atingiu 2,00 de 2,00

Dada uma lista encadeada, troque cada dois nós adjacentes e retorne o início da lista. Você deve resolver o problema sem modificar os valores nos nós da lista (ou seja, somente os próprios nós podem ser alterados).

Complete a função trocaPares para retornar a início da lista modificada.

Entrada

A entrada para seu programa é uma linha que contém uma lista com todos os nós da lista.

Saída

A saída deve conter a lista modificada.

Notas

- **Não** é necessário ler a entrada do usuário.
- Basta completar a função trocaPares

For example:

Input	Result
`[1,2,3,4]`	`[2,1,4,3]`
`[5,6,7,8,9]`	`[6,5,8,7,9]`
`[10,20,30]`	`[20,10,30]`

Answer: (penalty regime: 0, 0, 1, 2, ... %)

RESET ANSWER

```
// Considere essa estrutura pronta:
// struct ListNode {
//     int val;
//     struct ListNode *next;
// };

struct ListNode* trocaLista(struct ListNode* head){
    if (head == NULL || head->next == NULL) return head;

    struct ListNode* new_head = head->next;
    struct ListNode* anterior = NULL;
    struct ListNode* aux = head;

    while (aux != NULL && aux->next != NULL) {
        struct ListNode* next = aux->next;
        aux->next = next->next;
        next->next = aux;

        if (anterior != NULL) {
            anterior->next = next;
        }

        anterior = aux;
        aux = aux->next;
    }

    return new_head;
}
```

	Input	Expected	Got	
✔	`[1,2,3,4]`	`[2,1,4,3]`	`[2,1,4,3]`	✔
✔	`[5,6,7,8,9]`	`[6,5,8,7,9]`	`[6,5,8,7,9]`	✔
✔	`[10,20,30]`	`[20,10,30]`	`[20,10,30]`	✔

Passou em todos os teste! ✔

Correto

Notas para este envio: 2,00/2,00.

Questão 5

Correto

Atingiu 3,00 de 3,00

Dada uma string s contendo apenas os caracteres '(', ')', '{', '}', '[' e ']', determine se a string de entrada é válida.

Uma string de entrada é válida se:

- Os parênteses abertos devem ser fechados com o mesmo tipo de parênteses.
- Os parênteses abertos devem ser fechados na ordem correta.
- Todo parênteses fechado tem um parênteses aberto correspondente do mesmo tipo.

Entrada

A entrada para seu programa é uma linha que contém uma string com parênteses

Saída

O retorno da função deve ser 1 se a entrada for válida; e 0 se for inválida.

Notas

- **Não** é necessário ler a entrada do usuário.
- Basta completar a função isValid.
- Você tem à disposição uma pilha.
- Considere que o código abaixo já está todo implementado:

```
typedef struct Node {
    char value;
    struct Node* next;
} Node;

typedef struct Stack {
    Node* top;
} Stack;

Stack* createStack();

int isEmpty(Stack* stack);

void push(Stack* stack, char value);

char pop(Stack* stack);

char peek(Stack* stack);

void freeStack(Stack* stack);
```

For example:

Input	Result
`()`	1
`([])`	1
`(]`	0

Answer: (penalty regime: 0, 0, 1, 2, ... %)

RESET ANSWER

```
int isValid(char* s){
    Stack* parenteses = createStack();
    Stack* chaves = createStack();
    Stack* colchetes = createStack();

    for(int i = 0 ; s[i] != '\0' ; i++){
        if(s[i] == '('){
            push(parenteses, 'a');
        }
        else if(s[i] == ')'){
            if(!isEmpty(parenteses)){
                pop(parenteses);
            }
            else{
                return 0;
            }
```

```
        }
        else if(s[i] == '{'){
            push(chaves, 'a');
        }
        else if(s[i] == '}'){
            if(!isEmpty(chaves)){
                pop(chaves);
            }
            else{
                return 0;
            }
        }
        else if(s[i] == '['){
            push(colchetes, 'a');
        }
        else if(s[i] == ']'){
            if(!isEmpty(colchetes)){
                pop(colchetes);
            }
            else{
                return 0;
            }
        }
    }
    if(isEmpty(parenteses) && isEmpty(chaves) && isEmpty(colchetes)){
        return 1;
    }
    else{
        return 0;
    }
}
```

	Input	Expected	Got	
✔	`()`	1	1	✔
✔	`([])`	1	1	✔
✔	`(]`	0	0	✔

Passou em todos os teste! ✔

Correto

Notas para este envio: 3,00/3,00.